

Noé-Yann
CHAFFARD-KOWALEWSKI
BUT INFO - 2
IUT - St Dié des Vosges

Rapport de stage au LEAD

Table des matières :

1. Introduction et présentation de l'entreprise
2. Étude
 - a. Analyse des besoins et présentation des missions du stage
 - b. Présentation des solutions techniques possibles et justification de leur emploi
 - c. Méthodologie de travail (ressources, calendrier, organisation, etc.)
 - d. Mise en œuvre de la/des solutions retenues et résultats
3. Conclusion et perspectives personnelles
4. Annexes

I- Introduction :

Ce rapport traite de mon stage qui s'est tenu du 4 mars au 24 avril 2026 au LEAD à Dijon. Nous allons aborder successivement l'introduction de l'entreprise, l'analyse des besoins, les solutions techniques envisagées, la méthodologie de travail, la mise en œuvre des solutions choisies et les résultats obtenus, avant de conclure par des perspectives personnelles, complétées par des annexes.

Présentation de l'entreprise :

Mon stage s'est déroulé au LEAD, une unité de recherche affiliée au CNRS et à l'Université de Bourgogne. Le CNRS, ou Centre National de la Recherche Scientifique, est le plus grand organisme public français de recherche scientifique, couvrant tous les domaines de la connaissance.

LEAD signifie Laboratoire d'Étude de l'Apprentissage et du Développement. Il est situé sur le campus universitaire de Dijon. C'est une unité mixte de recherche qui étudie l'apprentissage humain depuis sa création dans les années 1980, et l'apprentissage artificiel depuis les années 2000.

L'axe neurosciences et psychologie cognitive se concentre sur les modifications des processus de traitement de l'information liées aux apprentissages (implicites, didactiques et professionnels), au développement de l'individu (enfant, personne âgée) et à ses perturbations, qu'elles soient dues à des handicaps (cécité, surdité) ou à des troubles neuropsychiques.

L'axe intelligence artificielle et informatique neuro-inspirée intègre l'IA dans divers domaines tels que la santé (pathologies du rein, cancer de l'ovaire, maladie d'Alzheimer, systèmes de substitution sensorielle), l'industrie (reconnaissance de défauts de pièces industrielles), le patrimoine (transcription de textes historiques) et la reconnaissance des émotions.

II- Étude

a) Analyse des besoins et présentation des missions du stage

Pour commencer, je vais présenter ce qu'est Skinet. Skinet est une application de segmentation et de traitement de biopsies rénales développée par l'axe intelligence artificielle du laboratoire. Une biopsie rénale est une image d'un fragment de rein prélevé. La segmentation est une technique de vision par ordinateur qui divise une image numérique en groupes distincts de pixels pour identifier et classer les objets ou régions. Elle est utilisée pour analyser la qualité d'un greffon rénal et pour étudier la maladie de Berger (un greffon rénal est le rein transplanté d'un donneur à un receveur lors d'une greffe). Actuellement, cette application fonctionne localement sur un serveur de calcul du LEAD. Elle est en cours de développement, ce qui influe sur mon stage. Pendant celui-ci, j'ai travaillé à partir d'une ancienne version de Skinet. Le fait que l'application soit en local sur un serveur de calcul complique son utilisation pour un médecin non initié à certaines bases de l'informatique.

La mission confiée par mon maître de stage était : "Réalisation d'une interface graphique pour une application de segmentation et de traitement de biopsies rénales". Plus précisément, je devais créer une interface graphique web accessible depuis un navigateur, même pour ceux qui ne sont pas informaticiens. Durant mon stage, j'ai travaillé exclusivement sur cette mission pendant 8 semaines. Pour comprendre les subtilités de la consigne, j'ai discuté avec un médecin collaborant avec les développeurs du projet. Ensuite, mon maître de stage m'a fourni une liste concrète d'objectifs à atteindre, et enfin, j'ai rédigé un cahier des charges pour formaliser ma compréhension et établir une première ligne directrice à suivre lors de ces huit semaines. Le contenu du cahier des charges est disponible en annexe n°1.

Je voyais ce stage comme une opportunité de découvrir le monde du travail et, en particulier, le monde de la recherche en laboratoire. C'était aussi l'occasion de m'améliorer en développement web full-stack et de réaliser un projet concret sur le sujet des API, abordé récemment en cours. Pour moi, travailler en pratique est toujours plus efficace que d'étudier uniquement la théorie, surtout en informatique. Mon objectif personnel pour la fin du stage était de fournir une interface web intégrant les fonctionnalités demandées, tout en acquérant de nouvelles compétences.

b) Présentation des solutions techniques possibles et justification de leur emploi

Avant de commencer à construire le site, il fallait réfléchir à son apparence. La première option était de créer une page par "mode" : une pour l'entraînement de l'intelligence artificielle, une pour l'inférence (terme désignant l'exécution d'une intelligence artificielle) et une pour la modification des résultats de l'inférence.

La deuxième option était de concevoir une page unique avec un navigateur de dossiers permettant de lancer le mode souhaité, avec un bouton principal pour lancer l'inférence et des zones pour afficher les résultats.

C'est cette dernière option qui a été retenue, bien que la première figure dans le cahier des charges. Après discussions avec l'équipe de projet, il s'est avéré qu'une interface unique serait plus simple d'utilisation. Le mode entraînement était trop complexe pour une interface graphique, et seules des personnes avec des connaissances informatiques de base l'utiliseraient. De plus, réaliser le mode modification en seulement deux mois s'avérait trop ambitieux, en plus du reste de la page web à développer. Enfin, Skinet étant en cours de développement, la première solution risquait de devenir obsolète rapidement, alors que la deuxième resterait fonctionnelle avec quelques ajustements du bouton principal, simplifiant ainsi la maintenance.

Après avoir déterminé la structure de l'interface, il fallait choisir les langages de programmation. Pour le frontend (partie visible avec laquelle l'utilisateur interagit directement), l'utilisation du HTML, du CSS et du JavaScript s'imposait. Pour le backend (partie serveur gérant la logique, les données et les traitements), notamment pour l'API (interface de communication permettant à une application web d'échanger avec le serveur Skinet), j'ai opté pour Python avec le framework FastAPI, apprécié pour sa simplicité d'utilisation et sa fiabilité.

c) Méthodologie de travail

Pour programmer, j'utilisais l'éditeur VS Code, que j'utilise déjà pour mes projets personnels et mes cours, m'évitant ainsi d'en apprendre un autre. Il fonctionne avec des extensions, ce qui m'a permis de coder depuis mon ordinateur tout en sauvegardant mon travail sur une session personnelle sur le serveur de calcul pour accéder à Skinet.

Comme mentionné, le programme de segmentation nécessite un serveur puissant pour les inférences. Pour y accéder, il faut utiliser un VPN spécifique (réseau privé

virtuel qui sécurise et masque la connexion) et se connecter à la machine via SSH (protocole sécurisé pour l'accès à distance). J'utilisais donc EduVPN pour la connexion sécurisée, X2GO pour les sessions graphiques, et Bitvise SSH pour le terminal et le port forwarding (rediriger un port distant vers un port local pour accéder à des services internes), couplé à l'extension VS Code "sftp sync" pour la synchronisation SFTP (protocole sécurisé de transfert de fichiers). J'utilisais deux petits serveurs locaux pour mon interface. Le premier, un serveur Python démarré avec `"python3 -m http.server 8000"`, fait tourner le frontend de la page web. Le deuxième, un serveur Uvicorn démarré par `"uvicorn api:app --reload --port 8080"`, exécute l'API avec efficacité (la gestion asynchrone permet au programme de continuer sans attendre la fin d'opérations longues). Voir l'annexe n°2.

Pour travailler de manière organisée, j'ai d'abord rédigé un cahier des charges, comme expliqué précédemment, puis planifié mon travail sur une petite page HTML locale, divisée en trois parties : tâches à réaliser, tâches hebdomadaires, et tâches quotidiennes, classées par priorité.

Pour assurer la qualité de mon travail, je communiquais régulièrement avec mon maître de stage, que ce soit en personne pour montrer mes avancées, par mail pour des réponses rapides, ou via des rapports hebdomadaires. Cela me permettait de suivre mes progrès. Je consultais aussi les autres membres du projet par mail ou directement au laboratoire.

d) Mise en œuvre de la/des solutions retenues et résultats

Voici ci-dessous, l'historique de la mise en œuvre du projet.

Semaine 1 & 2 -

Les trois premiers jours de stages ont été consacrés à la découverte du LEAD et à la lecture du rapport présentant le programme Skinet, afin de comprendre son rôle dans la recherche médicale et d'anticiper les besoins liés à la conception de l'interface graphique.

J'ai ensuite pris en main l'environnement de travail, hébergé sur un serveur distant accessible via SSH et nécessitant une connexion VPN. Cette étape m'a permis de comprendre l'infrastructure existante et les contraintes techniques associées.

Ensuite, j'ai travaillé sur l'élaboration du cahier des charges, en définissant les besoins fonctionnels, les contraintes techniques et les objectifs de l'interface.

Enfin, j'ai configuré les connexions nécessaires entre mon navigateur, mon éditeur de code et le serveur distant via SSH, notamment grâce à la mise en place d'un port forwarding et d'un outil de synchronisation SFTP. J'ai également commencé à étudier les frameworks susceptibles d'être utilisés pour le projet, notamment FastAPI.

Durant ces semaines, j'ai eu du mal à identifier clairement toutes les entrées et sorties que l'interface devait gérer pour rédiger le cahier des charges. J'ai pris le temps d'explorer le programme en détail et j'ai demandé des précisions à mon maître de stage, ce qui m'a permis d'y voir plus clair et d'avancer.

J'ai aussi rencontré des difficultés pour faire fonctionner les connexions entre mon ordinateur, l'éditeur de code et le serveur. Là encore, en prenant le temps de comprendre comment tout s'articule et en testant différentes configurations, j'ai fini par réussir à mettre en place une solution stable.

Semaine 3 -

Durant la première partie de la semaine, j'ai étudié le fonctionnement de FastAPI, Conda et Uvicorn. J'ai créé un petit environnement local de test pour pratiquer ce que j'ai appris, en établissant une connexion entre une page HTML et une API.

La deuxième partie de la semaine s'est résumée à déployer sur le serveur ce que j'avais réussi à faire en local.

Durant cette semaine, la plus grande difficulté était de réussir à appeler l'API depuis le JS sur le serveur. Même s'ils sont dans le même dossier, vu que le site web est affiché sur le navigateur de l'utilisateur connecté via un Local Port Forwarding (Redirection de port local en français), le site web s'attend à ce que le serveur Uvicorn le soit aussi. J'ai donc dû connecter le serveur Uvicorn sur un autre Local Port Forwarding, ce qui m'a pris du temps à comprendre et du temps à mettre en place.

Par ailleurs, j'ai mis plus de temps à maîtriser l'environnement et les connexions au serveur que ce que j'avais imaginé et prévu, je n'ai donc réussi à faire une connexion entre le HTML-JS et l'API que vendredi soir. Ce n'est pas une difficulté en soi mais la gestion et la prévision du temps est un exercice où j'ai encore beaucoup d'améliorations à faire.

Semaine 4 & 5 -

La première semaine a été écourtée en raison d'un arrêt maladie (deux jours). Sur les trois jours de présence, j'ai travaillé sur l'apprentissage de subprocess et la

manipulation de fichiers texte. Subprocess est une bibliothèque Python qui permet d'exécuter des commandes dans un terminal séparé, puis d'en récupérer les retours. Dans mon projet, je l'utilise pour exécuter le programme Skinet depuis l'API. Ensuite, je récupère les logs (messages enregistrés par le programme pour décrire ce qui se produit pendant son exécution) dans un fichier texte.

Cependant, durant la semaine, j'ai surtout lu de la documentation et fait des tests sur des fichiers à part, mais sans résultats.

Je n'ai réussi qu'en début de semaine suivante à faire fonctionner les fonctionnalités, que j'ai divisées en deux boutons sur l'interface, avec deux appels API distincts : un qui exécute Skinet et l'autre qui récupère les logs enregistrés dans un fichier texte et les affiche.

J'ai passé le milieu de la deuxième semaine à nettoyer le code en mettant des noms de fonctions et de variables explicites, à mettre en place un dépôt Git et à structurer le projet en différents dossiers afin de séparer le frontend et le backend. J'ai aussi rendu les logs plus lisibles. Ils étaient initialement affichés sous la forme d'un seul bloc qui n'allait à la ligne qu'à l'extrémité de la page, en affichant « \n » au lieu de revenir à la ligne comme à l'original.

De plus, au lieu d'afficher une seule barre de chargement, toutes les étapes par lesquelles elle était passée étaient affichées. J'ai donc fait en sorte qu'à chaque occurrence de « \n », le texte aille à la ligne, et je n'ai laissé visibles que les barres de chargement indiquant 100 %. J'ai enfin ajouté un peu de style afin d'afficher les logs en blanc sur fond noir pour indiquer visuellement qu'il s'agit d'un retour de terminal.

J'ai passé mon dernier jour à créer un script Bash qui initialise un serveur web local pour les fichiers statiques (HTML et JS), active l'environnement Conda du projet, puis démarre l'API FastAPI afin d'assurer le fonctionnement cohérent de l'interface et du backend en local.

Ce script permet d'éviter des erreurs incompréhensibles pour des non-informaticiens lorsqu'un port du serveur est déjà utilisé.

Durant ces semaines, j'ai rencontré des difficultés que je n'avais pas prévu. En effet, lorsque j'ai développé l'API et le JS, j'ai commencé à tout faire dans le même dossier et à utiliser des noms de variables peu compréhensibles. Rendre l'ensemble propre m'a pris un temps que j'aurais pu utiliser pour avancer sur le projet.

Comme je ne connaissais pas du tout subprocess, comprendre le fonctionnement de la bibliothèque ainsi que ses possibilités et ses limites m'a demandé un certain temps de tests et de lecture de documentation.

Semaine 6 -

En début de semaine, j'ai développé un appel API qui permet d'obtenir le contenu du dossier et des sous-dossiers de Skinet, ainsi qu'un autre qui permet de récupérer une image contenue dans celui-ci.

Pour la récupération du contenu des dossiers, je prends en entrée le chemin pour accéder au dossier depuis /Skinet, puis, à l'aide de subprocess, j'exécute la commande "ls".

Pour la récupération des images, je prends aussi en entrée le chemin qui mène à l'image dans un des sous-dossiers de Skinet et je retourne l'image à l'aide de la fonction fournie par FastAPI, FileResponse(). Elle permet de retourner autre chose que du JSON.

En seconde partie de semaine, j'ai codé en JS le moyen de naviguer dans le dossier Skinet depuis la page web et d'afficher les images à l'aide des appels API codés précédemment. Il est possible d'aller dans le sous-dossier suivant, de revenir au dossier parent tant que l'on n'est pas dans /Skinet et d'afficher les images.

En fin de semaine, j'ai essayé de faire en sorte de pouvoir modifier le port d'Uvicorn. Désormais, lancement.sh crée un fichier JSON contenant le numéro de port entré via les paramètres lors de son exécution. Cependant, je n'arrive pas à récupérer ce contenu en JS pour l'intégrer dans les liens lors des appels API. Cela est dû à la différence entre les fonctions synchrones et asynchrones. Je traiterai cela lundi.

Semaine 7 -

En début de semaine, j'ai terminé de coder la possibilité de se connecter au serveur Uvicorn pour effectuer une requête d'API depuis un port personnalisé. La problématique venait du fait que le port est stocké dans un fichier lorsqu'il est défini en exécutant ./lancement.sh, et donc récupéré via une fonction asynchrone en JavaScript, ce qui empêche de le stocker dans une variable de session qui, elle, est évaluée de manière synchrone. J'ai donc implémenté une fonction asynchrone qui est appelée à chaque requête API, prenant en entrée le chemin de l'appel et un endpoint (paramètre) ou non, qui va chercher le numéro de port, puis retourne, sous la forme d'une chaîne de caractères, le lien complet de l'appel sous la forme "http://localhost:port/mon-appel/parametre". Cela ne pose pas de problème car, pour faire un appel, on utilise la méthode fetch(), qui est asynchrone, ce qui implique que les fonctions faisant un appel sont déjà asynchrones.

Cette manière de faire implique qu'il y a deux requêtes à chaque fois, ce qui ralentit la réponse finale. C'est particulièrement visible dans la zone de navigation du dossier Skinet. Cependant, c'est la seule manière de faire que j'ai trouvée. Suite à cela, j'ai mis à jour le README.

Ensuite, en milieu de semaine, j'ai conçu l'affichage des fichiers JSON, pour pouvoir modifier les fichiers de configuration depuis l'interface . L'affichage se fait dans un <textarea>, qui est natif en HTML, ce qui facilite la modification du texte.

En fin de semaine, j'ai codé la possibilité d'enregistrer ce que l'utilisateur a modifié, dans son fichier d'origine. Cela m'a pris plus de temps que je ne l'avais prévu car c'était la première fois que j'utilisais l'API dans le sens utilisateur → serveur ; jusque-là, l'API servait uniquement à récupérer des données depuis le serveur.

J'ai aussi implémenté une popup qui s'affiche lorsque l'utilisateur s'apprête à enregistrer la modification du JSON affiché pour lui demander confirmation, ainsi que pour lui rappeler de sauvegarder une copie s'il le souhaite.

Semaine 8 -

En début de semaine, j'ai nettoyé le code de fonctions plus utile après l'ajout de certaines fonctionnalités et ai ajouté des commentaires afin de rendre mon code plus compréhensible pour une future personne qui pourrait travailler sur mon projet.

J'ai ensuite ajouté la possibilité d'arrêter une inférence en cours et ai rendu plus clair le signalement de l'état de l'inférence (en cours, fini, interrompu et erreur durant l'inférence).

En milieu de semaine, j'ai ajouté la possibilité de télécharger un fichier dans le dossier du navigateur où l'utilisateur se trouve.

Enfin, j'ai réorganisé le visuel de l'interface web en CSS pour que l'utilisation soit instinctive pour un utilisateur ne connaissant pas bien l'interface.

Ce stage m'a permis d'apprendre de nouvelles compétences.

- Découverte du milieu professionnel : c'est ma première expérience en entreprise, ce qui m'a permis de comprendre les attentes, les méthodes de travail et l'organisation d'un environnement de recherche. Travail sur une infrastructure distante : j'ai appris à utiliser efficacement les outils nécessaires pour me connecter à un serveur distant, à naviguer dans un environnement Linux et à comprendre les implications d'un développement sur machine distante.
- Conception d'un cahier des charges : la réalisation d'un document plus complet que ceux réalisés en BUT m'a permis d'améliorer ma capacité à structurer, hiérarchiser et formaliser les informations.
- Mise en place d'outils de développement : j'ai appris à utiliser l'extension SFTP de VS Code pour synchroniser mon code avec le serveur, ainsi qu'à configurer un port forwarding SSH permettant d'accéder au serveur web depuis mon navigateur.

- Découverte de FastAPI : j'ai commencé à me familiariser avec ce framework Python, qui est utilisé pour développer l'API de l'interface web.
- Utilisation d'un environnement Conda : C'est la première fois que je travaille dans ce genre d'environnement, ça m'a permis de comprendre concrètement à quoi cela sert et à quel point c'est utile.
- J'ai appris à utiliser Fast API et Uvicorn.
- J'ai aussi appris et compris plus en détail comment fonctionnent les connexions entre l'ordinateur et le serveur ainsi que l'importance d'un Local Port Forwarding.
- Utilisation de subprocess et manipulation de fichiers texte.
- Écriture de code propre et utilisation de Git. J'ai, pour la première fois, réalisé des commits sur différentes branches ainsi que des pull-requests.
- Programmation en Bash. J'ai pu améliorer mes compétences dans ce langage, en faisant un programme qui a du sens dans un projet.
- Utilisation de paramètres dans les appels API.
- Retour des API autre que par JSON.
- Les fonctions synchrones et asynchrones en JS.
- Manipulation des <textarea> en html.
- Manipulation de requête comportant des données dans le corps (json)
- Affichage d'une pop-up
- Uploader un fichier en JS avec fastAPI.

Afin de comprendre en détail les tenants et aboutissants du travail réalisé, il y a un schéma expliqué de l'architecture de l'interface web à l'annexe n°3.

Conclusion

Le stage effectué au sein du LEAD, une unité de recherche du CNRS à l'Université de Bourgogne, a permis de travailler sur le projet Skinet, une application dédiée à la segmentation et au traitement de biopsies rénales. Mon principal objectif était de développer une interface graphique web accessible pour les professionnels de santé, simplifiant ainsi l'utilisation de Skinet au-delà des compétences informatiques nécessaires.

Durant ces huit semaines, j'ai acquis des compétences essentielles en développement web full-stack, notamment en utilisant HTML, CSS, JavaScript pour le frontend et Python avec FastAPI pour le backend. J'ai également amélioré mes connaissances en gestion de serveurs distants, VPN, et en configuration de connexions sécurisées. J'ai rencontré et surmonté diverses difficultés, notamment dans la gestion de l'environnement distant et la compréhension des bibliothèques comme subprocess.

Cette expérience m'a permis de me familiariser avec le monde professionnel, particulièrement dans un contexte de recherche, tout en consolidant mes connaissances théoriques par la pratique. Elle m'a aussi posé des défis de gestion du temps et de priorisation des tâches, compétences que je compte approfondir à l'avenir.

En perspective, ce stage m'encourage à poursuivre dans le développement d'applications concrètes qui répondent aux besoins des utilisateurs finaux. Travailler sur ce projet me motive à poursuivre ces études et approfondir cette voie.

Annexes

Annexe n°1 - Contenu du cahier des charges

Présentation du Projet

- Skinet est une application de segmentation et de traitement de biopsies rénales.
- Actuellement, pour exécuter le programme, il faut le faire en ligne de commande via un environnement sur le serveur de calcul. Créer une interface graphique permet à un non-développeur d'utiliser l'outil plus simplement.
- L'objectif de cette interface est d'exécuter le code à partir de données renseignées dans des formulaires, via un bouton d'exécution, puis d'afficher les résultats produits par Skinet.

Analyse Fonctionnelle

- Choix du mode (Entraînement, Inférence, Modification).
- Édition des fichiers de configuration.
- Passage des paramètres d'entrée.
- Lancement du programme existant via un bouton.
- Affichage des traces logicielles pendant l'exécution.
- Modification des segmentations à partir d'une image et d'un fichier de résultat de segmentation.
- Téléchargement des résultats.

Analyse Technique

- **Environnement actuel** : Actuellement, après s'être connecté au serveur via SSH, en passant par le VPN, il faut ajouter manuellement les fichiers dans le bon répertoire, puis exécuter le script à l'aide d'une commande.
- **Technologie** : Afin de permettre la communication entre la page web et le programme Python, il faut créer une API. Le framework choisi pour cela est FastAPI, un framework Python, en raison de sa fiabilité et de sa simplicité d'utilisation.

Pour le frontend, je vais utiliser HTML, CSS et JS.

Chaque mode sera paramétré sur une page différente.

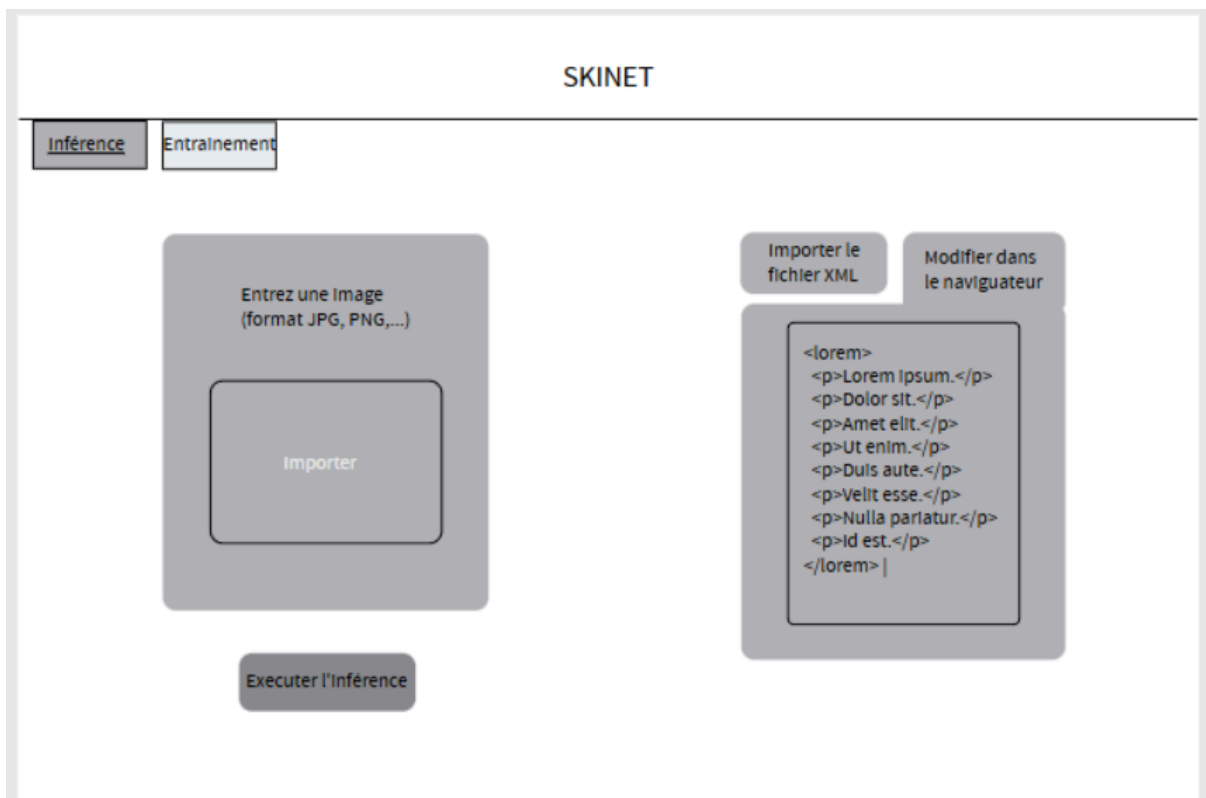
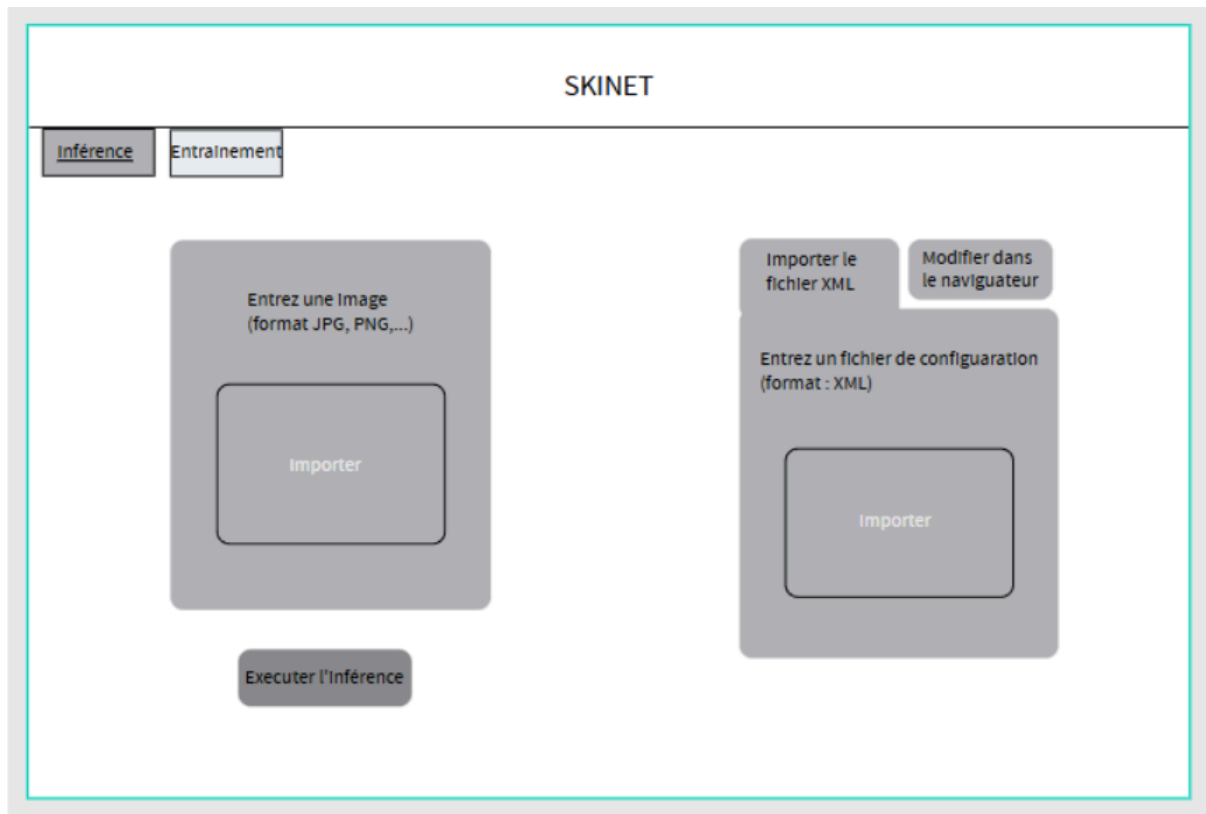
Pour le mode inférence : L'édition des fichiers de configuration se fera via une zone de modification textuelle du XML ou via une zone permettant d'importer un fichier XML. Là où les images seront chargées depuis un formulaire de type sélecteur de fichiers, spécialisé pour les images (PNG, JPG, etc.).

Pour le mode entraînement : Un formulaire accueillera les fichiers dont le programme a besoin (image d'entraînement et son annotation).

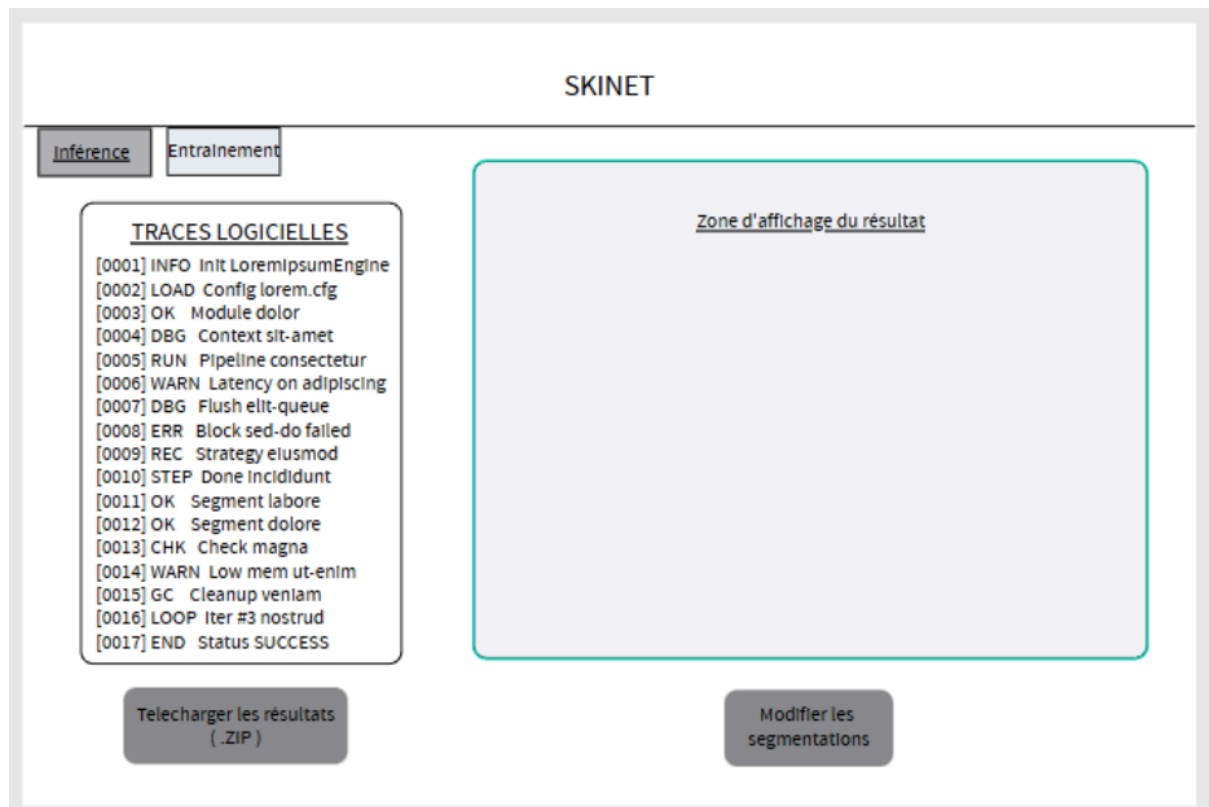
Chaque exécution ou confirmation se fera via un bouton. L'affichage des traces logicielles se fera via une zone de texte qui se met à jour dynamiquement.

Interface Utilisateur (UI/UX)

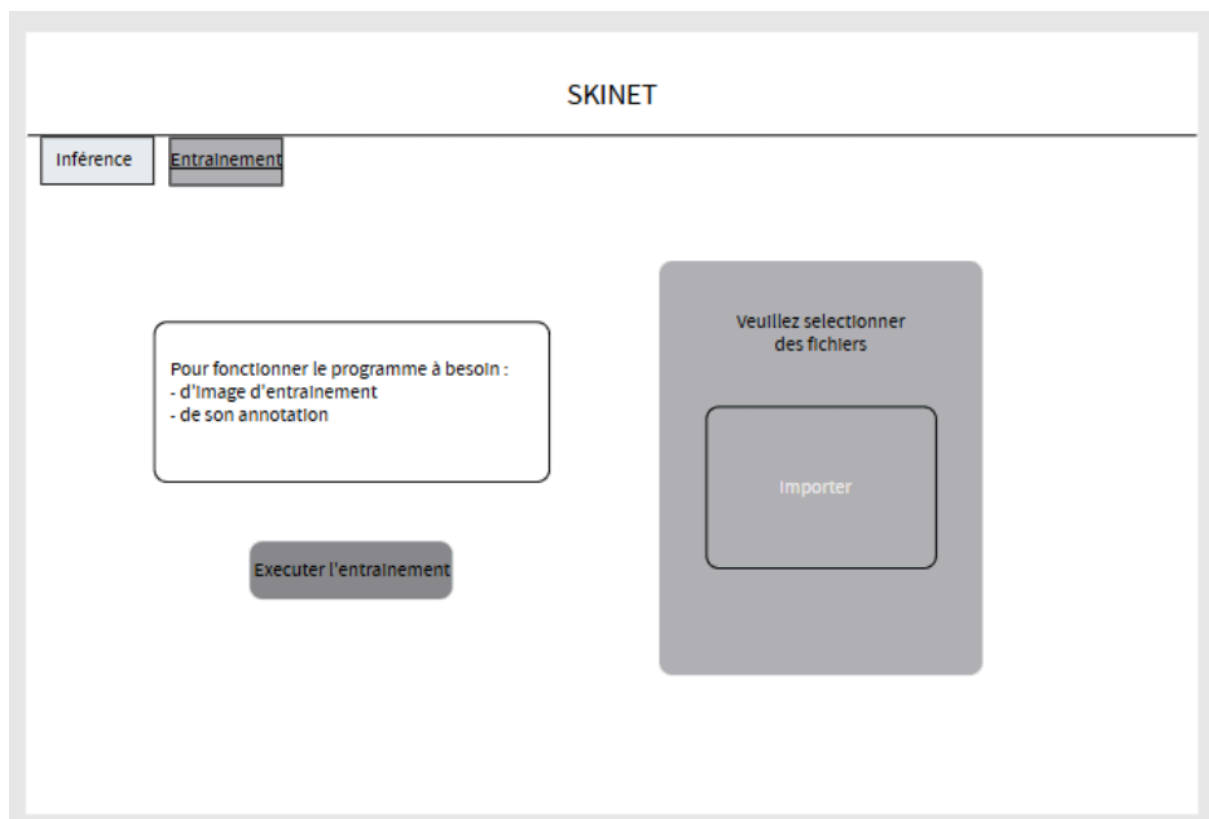
Maquettes de l'interface du mode inférence :



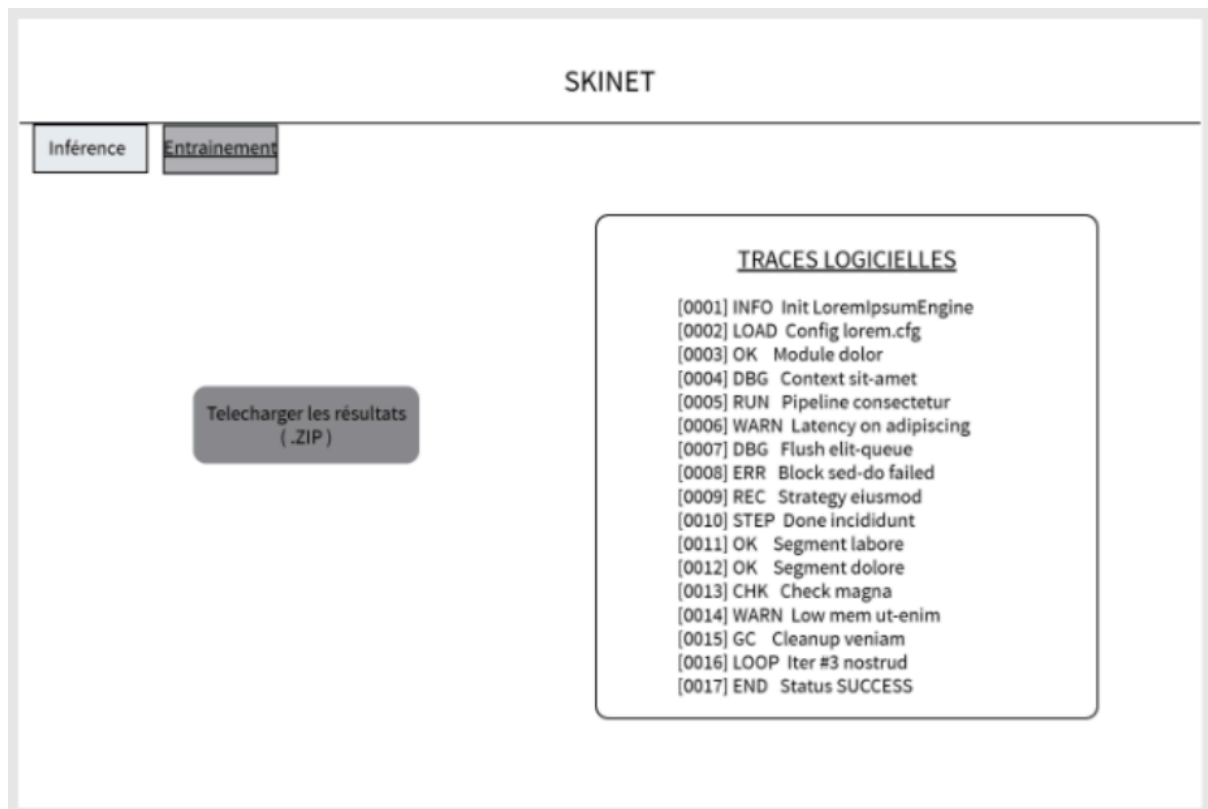
Maquette de l'interface du mode inférence une fois exécutée :



Maquettes de l'interface du mode entraînement:



Maquette de l'interface du mode entraînement une fois exécutée :



Sécurité

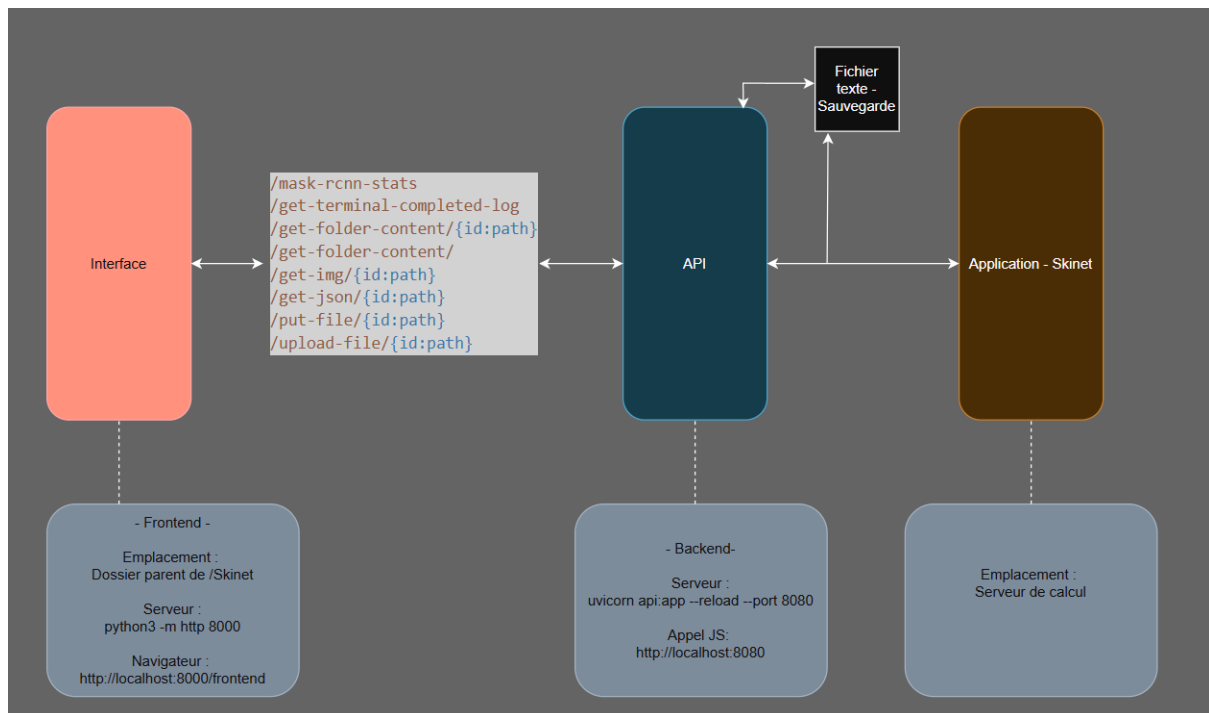
Sécurisation des entrées, particulièrement de la modification des fichiers de config xml.

- **Gestion des erreurs** : La gestion des erreurs se fait via le panneau de traces logicielles, qui affiche les messages d'erreur.
- **Validation des données** : Les entrées utilisateur devront être validées selon des règles explicites de format, de type, de taille et de contenu. Toute donnée non conforme sera rejetée avant traitement.

Annexe n°2 - Configuration des ports dans bitwise SSH

Login	Options	Terminal	RDP	SFTP	Services	C2S	S2C	SSH	Notes	About
	Listen interface	Port	Destination host	Port	Comment					
☒	127.0.0.1	8000	localhost	8000	Pour connection navigateur - serv					
☒	127.0.0.1	2222	localhost	22	Pour connection VScode - serv					
☒	127.0.0.1	8080	localhost	8080	Pour la connexion avec l'api					

Annexe n°3 - Schéma de l'architecture de l'interface web



Annexe n°4 - Capture d'écran de l'interface web

- SKINET -

